# Benchmark Report —Search Methods Part 2

## Methods Implemented

- All algorithms implemented as Python generators, yielding state at each step to drive the visualizer
- BFS and DFS use a deque/stack; IDDFS uses recursive depth-limited search with iterative deepening
- Greedy and A* use a min-heap through heapq
- Path reconstruction done through parent dictionary traceback

## Heuristics

Grid world:

- Manhattan: admissible for 4-way movement
- Euclidean: admissible for 8-way movement
- Chebyshev: admissible for 8-way (diagonal)

City graph:

- Euclidean (coordinate lookup): straight-line distance via lat/lon
- Haversine: great-circle Earth distance in km

## Experimental Setup

| Setting | Grid Size | Obstacle Density | Connectivity | Runs |
|---|---|---|---|---|
| Easy | 8×8 | 20% | 4-way | 5 |
| Medium | 15×15 | 25% | 4-way | 5 |
| Hard | 25×25 | 30% | 4-way | 5 |

Seeds:

- Easy: [3212, 6559, 7559, 455, 2040]
- Medium: [981, 3512, 1183, 7370, 7565]
- Hard: [2789, 7328, 4297, 7941, 3790]

Heuristic used: Euclidean.

- Memory measured through `tracemalloc` (process peak).
- Time through `time.perf_counter`.

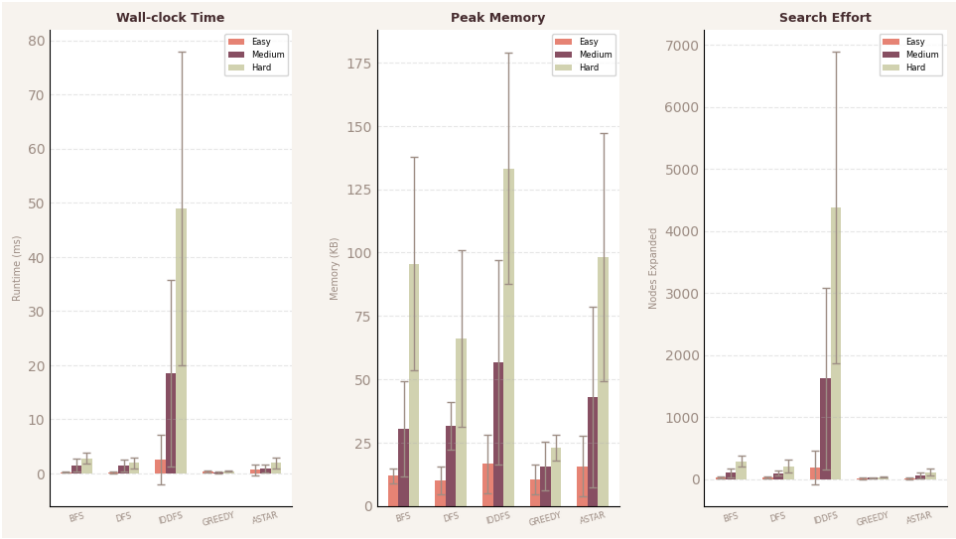
**Results**

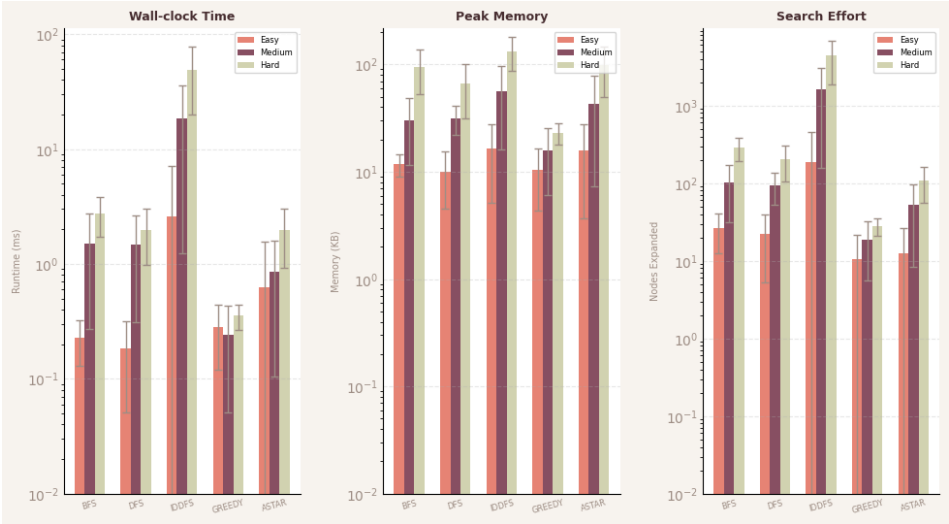
```
PS C:\Users\ahmed\source\repos\CS449_SearchMethods_Pt2> python main.py

============================================================
  BATCH BENCHMARK — EASY (5 runs)
  Seeds: [9566, 1058, 6628, 2322, 5444]
  Heuristic: euclidean
============================================================

Algo        Solved   Time(ms)        Mem(KB)        NodesExp      Cost        Optimal
--------------------------------------------------------------------------------------
bfs            5      0.23±0.10      11.85±2.80       26.6±14.0       6.40±3.71    ✓
dfs            5      0.19±0.13      10.02±5.50       22.4±17.1      11.60±6.54
iddfs          5      2.58±4.59      16.56±11.43     187.4±272.4      7.60±6.39
greedy         5      0.28±0.16      10.36±5.98       10.6±10.9       6.40±3.71    ✓
astar          5      0.63±0.94      15.67±11.98      12.4±13.8       6.40±3.71    ✓

============================================================
  BATCH BENCHMARK — MEDIUM (5 runs)
  Seeds: [7094, 1673, 301, 2032, 6180]
  Heuristic: euclidean
============================================================

Algo        Solved   Time(ms)        Mem(KB)        NodesExp      Cost        Optimal
--------------------------------------------------------------------------------------
bfs            5      1.51±1.24      30.39±18.71     101.0±70.0      13.60±8.17    ✓
dfs            5      1.48±1.16      31.64±9.47       93.4±40.7      38.40±17.57
iddfs          5     18.52±17.28     56.73±40.44    1617.4±1462.2     19.60±13.45
greedy         5      0.24±0.19      15.72±9.69       18.8±13.3      15.20±9.96
astar          5      0.86±0.75      43.02±35.75      53.0±44.6      13.60±8.17    ✓
```

```
============================================================
  BATCH BENCHMARK — HARD (5 runs)
  Seeds: [1572, 1816, 1011, 1213, 7714]
  Heuristic: euclidean
============================================================
```

| Algo | Solved | Time(ms) | Mem(KB) | NodesExp | Cost | Optimal |
|---|---|---|---|---|---|---|
| bfs | 5 | 2.77±1.04 | 95.67±42.23 | 287.8±93.1 | 24.00±6.20 | ✓ |
| dfs | 5 | 1.99±1.01 | 66.11±34.93 | 205.2±100.9 | 52.00±20.24 | |
| iddfs | 5 | 48.98±28.98 | 133.27±45.66 | 4375.8±2514.2 | 37.20±14.92 | |
| greedy | 5 | 0.36±0.09 | 23.06±5.14 | 28.0±6.9 | 24.80±6.91 | |
| astar | 5 | 1.96±1.04 | 98.23±48.92 | 108.8±52.6 | 24.00±6.20 | ✓ |

## Chart



*A logarithmic scale was used for visualization to account for the significantly higher node expansion and runtime of IDDFS to have easier comparison across all algorithms.*

**Analysis**

1. **Overall Performance:**
    a. BFS and A* consistently gave optimal solutions with the lowest path cost
    b. DFS and IDDFS often found suboptimal paths. This trend can be seen as the difficulty and grid size increased, the performance decreased.
    c. Greedy search performed very efficiently and fastest overall, but it sacrificed optimality.
2. **Runtime:**
    a. For easy and medium grids, all algorithms performed similarly because the search space was pretty small.
    b. However, as difficulty increased-
        i. IDDFS increased in runtime significantly
        ii. BFS and A* scaled moderately but were still manageable
        iii. Greedy remained the fastest due to limited exploration
    c. Note - The log-scale chart clearly shows IDDFS growing disproportionately, confirming its poor scalability.
3. **Memory Usage**
    a. A* and BFS used the most amount of memory because of the large frontier and visited structures it had to maintain
    b. DFS and Greedy were the most memory efficient.
        i. DFS uses a minimal stack space
        ii. Greedy limits exploration using heuristics
    c. IDDFS shows moderate-to-high memory usage due to the repeated re-expanding of nodes
    d. This shows that there is a tradeoff between optimality and memory usage
4. **Search Effort**
    a. IDDFS has by far the highest expansion of nodes
        i. ~1600 (medium) and ~4300 (hard)
        ii. Caused by repeated exploration at increasing depth limits
    b. BFS also expanded many nodes, but fewer compared to IDDFS
    c. A* had reduced node expansion compared to BFS
    d. Greedy had the lowest node expansion because it focused directly on the goal
5. Solution Quality – Cost & Optimality
    a. BFS and A* found optimal paths consistently

   **b.** Greedy had optimal in easier cases but slightly worse in harder grids
   **c.** DFS and IDDFS produced significantly higher path costs and were less reliable as difficulty increased

**6.** Scalability
   **a.** As grid size and obstacle density increased**:**
      **i.** Uninformed searches (BFS, DFS, IDDFS) scaled poorly
      **ii.** Heuristic-based methods (Greedy, A*) scaled significantly better
      **iii.** IDDFS became the least practical algorithm due to exponential re-exploration

**7. Summary**
   **a.** Best overall: A* - optimal, efficient, scalable
   **b.** Fastest: Greedy – lowest run time and node expansion but not optimal
   **c.** Most inefficient: IDDFS – extremely high node expansion and runtime due to repeated work
   **d.** Baseline: BFS – always optimal but costly in memory and expansions